

Padrão Builder

"Separar a construção de um objeto complexo da sua representação, de modo que o mesmo processo de construção possa criar diferentes representações."

— Gang of Four, *Design Patterns: Elements of Reusable Object-Oriented Software*, 1994

DISCIPLINA

Padrões de Projeto de
Software

INSTITUIÇÃO

Universidade de Fortaleza —
UNIFOR

EQUIPE

Gustavo Q.
Mateus

APLICADO EM

AgroPlace — Backend Java 17 / Spring
Boot 3

SOBRE O PADRÃO

O que é o Padrão Builder?

Classificação GoF: Padrão Criacional (Creational Pattern)
Definido originalmente por Gamma, Helm, Johnson e Vlissides no livro de 1994.

O Builder separa o **algoritmo de construção** de um objeto complexo da sua **representação final**. Isso permite que o mesmo processo de montagem produza diferentes variantes de um objeto.

Diferente de uma *factory* que entrega o objeto em uma única chamada, o Builder monta o objeto **passo a passo** — cada etapa configura uma parte específica — e só materializa o produto ao chamar `construir()`.

Factory Method

Delega a criação a subclasses

Builder

Monta passo a passo
← **nosso foco**

Abstract Factory

Famílias de objetos

OS 4 PARTICIPANTES GOF

Builder

Interface que declara os métodos de configuração para cada parte do produto. Define o contrato de construção.

ConcreteBuilder

Implementa a interface Builder, acumula o estado interno e executa validações em `construir()`.

Director

Orquestra a sequência de chamadas ao Builder. Conhece o processo, não o produto resultante.

Product

O objeto complexo sendo construído. Seu construtor é restrito — só pode ser criado via Builder.

QUANDO APLICAR

Casos de uso do Padrão Builder

01

Objetos com muitos parâmetros

Quando um objeto exige 5 ou mais parâmetros no construtor, o Builder substitui a lista confusa por uma API fluente e nomeada.

02

Parâmetros opcionais

Evita a passagem explícita de null para campos opcionais. Apenas os métodos necessários são chamados.

03

Validação centralizada

O método `construir()` é o único ponto de validação — garante que nenhum objeto inválido seja criado antes da persistência.

04

Múltiplas representações

O mesmo Director pode usar diferentes ConcreteBuilders para produzir variantes do mesmo tipo de produto sem alterar o processo.

EXEMPLOS NO ECOSSISTEMA JAVA

`StringBuilder` · `HttpClient.newBuilder()` · `Stream.builder()` ·
`Locale.Builder`

EXEMPLOS NO SPRING

`UriComponentsBuilder` · `MockMvcRequestBuilders` ·
`SecurityFilterChain` via lambda

O PROBLEMA

Código *sem* Builder — o anti-padrão no AgroPlace

Como ficaria a entidade Anuncio e sua criação **sem o padrão Builder**:

ANUNCIO.JAVA — CONSTRUTOR TELESCÓPICO

```
public class Anuncio {  
  
    public Anuncio( // ← construtor público, 12 parâmetros  
        ContaUsuario conta,  
        String titulo,  
        String especie,  
        String raca,          // ← opcional, mas obrigatório aqui  
        int quantidade,  
        BigDecimal pesoMedioKg,  
        Integer idadeMediaMeses,  
        SexoAnimal sexo,  
        BigDecimal precoUnitario,  
        String cidade,  
        String estado,  
        String descricao) { // ← opcional, mas obrigatório  
        this.conta = conta;  
        // ... 11 atribuições sem nenhuma validação  
    }  
}
```

Uso no serviço de aplicação — **ilegível e inseguro**:

SERVICOANUNCIOPADRAO.JAVA — SEM BUILDER

```
Anuncio anuncio = new Anuncio(  
    conta,  
    req.titulo(),  
    req.especie(),  
    req.raca(),          // ← pode ser null?  
    req.quantidade(), // ← parâmetro 5 ou 6?  
    req.pesoMedioKg(),  
    req.idadeMediaMeses(),  
    req.sexo(),  
    req.precoUnitario(), // ← e se vier negativo?  
    req.cidade(),  
    req.estado(),       // ← e se tiver 3 chars?  
    req.descricao()  
); // ← zero validação antes do save()  
  
repositorio.save(anuncio); // ← objeto pode ser inválido
```

- ✗ Ordem dos 12 parâmetros causa bugs silenciosos
- ✗ Campos opcionais forçam null explícito na chamada
- ✗ Nenhuma validação — objeto inválido chega ao banco
- ✗ Construtor público expõe a criação a qualquer código

Código *com* Builder — como o AgroPlace resolve

ConcreteBuilder — acumula campos e valida tudo antes de criar o objeto:

ANUNCIOCONCRETOBUILDER.JAVA — CONSTRUIR() + VALIDAR()

```
@Override
public Anuncio construir() {
    validar(); // ← único ponto de validação
    return new Anuncio(this); // construtor package-private
}

private void validar() {
    if (conta == null)
        throw new IllegalStateException("Conta obrigatória");
    if (titulo == null || titulo.isBlank())
        throw new IllegalStateException("Título obrigatório");
    if (quantidade < 1)
        throw new IllegalStateException("Mínimo 1 animal");
    if (precoUnitario.compareTo(BigDecimal.ZERO) <= 0)
        throw new IllegalStateException("Preço deve ser positivo");
    if (estado.length() != 2)
        throw new IllegalStateException("Estado inválido (use UF)");
}
```

Director + Serviço — código limpo, sem conhecer os campos:

SERVICOANUNCIOPADRAO.JAVA — COM BUILDER

```
@Override
public RespostaAnuncio criar(
    ContaUsuario conta,
    RequisicaoAnuncio req) {

    DiretorAnuncio diretor =
        new DiretorAnuncio(
            new AnuncioConcretoBuilder() // ← injeção da impl
        );

    Anuncio anuncio =
        diretor.construirDaRequisicao(conta, req);
    // ↑ se inválido, lança exceção aqui — antes do save

    return montarResposta(
        repositorio.save(anuncio) // ← sempre válido
    );
}
```

- ✓ Campos nomeados — nenhuma ambiguidade de ordem
- ✓ Campos opcionais (raça, descrição) simplesmente omitidos
- ✓ Validação garante: objeto inválido nunca chega ao banco
- ✓ Construtor de Anuncio é package-private — blindado

Builder em 3 módulos do backend

MÓDULOS COM BUILDER

autenticacao

ContaUsuario — 7 campos: nome, email, telefone, documento, nomeEmpresa, senhaHash, tipoConta

anuncio

Anuncio — 12 campos: espécie, raça, peso, idade, sexo, quantidade, preço, cidade, estado, descrição, conta, título

perfil

PerfilProdutor, PerfilTransportadora e PerfilCorporativa — 3 builders distintos para 3 tipos de usuário

15

arquivos Builder

5

interfaces Builder

5

ConcreteBuilders

5

Directors

FLUXO COMPLETO — CRIAÇÃO DE ANÚNCIO

ControladorAnuncio → ServicoAnuncioPadrao → DiretorAnuncio
→ AnuncioConcretoBuilder → Anuncio

DIRETORANUNCIO.JAVA

```
public class DiretorAnuncio {

    private final AnuncioBuilder builder;
    // ↑ depende da interface, não da implementação

    public DiretorAnuncio(AnuncioBuilder b) {
        this.builder = b;
    }

    public Anuncio construirDaRequisicao(
        ContaUsuario conta,
        RequisicaoAnuncio req) {
        return builder
            .conta(conta).titulo(req.titulo())
            .especie(req.especie()).raca(req.raca())
            .quantidade(req.quantidade())
            .pesoMedioKg(req.pesoMedioKg())
            .sexo(req.sexo())
            .precoUnitario(req.precoUnitario())
            .cidade(req.cidade()).estado(req.estado())
            .descricao(req.descricao())
            .construir(); // ← valida e cria
    }
}
```

ContaUsuarioBuilder e uso no serviço

CONTAUSUARIOBUILDER.JAVA — INTERFACE

```
public interface ContaUsuarioBuilder {  
    ContaUsuarioBuilder nome(String nome);  
    ContaUsuarioBuilder email(String email);  
    ContaUsuarioBuilder telefone(String tel);  
    ContaUsuarioBuilder documento(String doc);  
    ContaUsuarioBuilder nomeEmpresa(String ne);  
    ContaUsuarioBuilder senhaHash(String sh);  
    ContaUsuarioBuilder tipoConta(TipoConta t);  
    ContaUsuario construir();  
}
```

TIPOCONTA.JAVA — ENUM

```
public enum TipoConta {  
    PRODUTOR,  
    TRANSPORTADORA,  
    CORPORATIVA  
}
```

SERVICOAUTENTICACAOPADRAO.JAVA — CADASTRAR()

```
DiretorContaUsuario diretor =  
    new DiretorContaUsuario(  
        new ContaUsuarioConcretoBuilder()  
    );  
  
// nomeEmpresa é opcional para PRODUTOR  
String nomeEmpresa =  
    requisicao.tipoConta() == TipoConta.PRODUTOR  
    ? null  
    : requisicao.nomeEmpresa();  
  
ContaUsuario conta =  
    diretor.construirConta(  
        requisicao.nome(),  
        requisicao.email(),  
        requisicao.telefone(),  
        requisicao.documento(),  
        nomeEmpresa,  
        codificadorSenha.encode(requisicao.senha()),  
        requisicao.tipoConta()  
    ); // ← validação executada internamente  
  
return montarResposta(repositorio.save(conta));
```

Três builders para três perfis de usuário

TRÊS PERFIS, TRÊS BUILDERS INDEPENDENTES

PerfilProdutorBuilder

nomePropriedade, CAR, documentoRural, inscricaoEstadual, areaHectares, cep, endereco, cidade, estado, descricao

PerfilTransportadoraBuilder

razaoSocial, CNPJ, RNTRC, tiposVeiculo, capacidadeCargaKg, possuiTransporteVivo, responsavelOperacional

PerfilCorporativaBuilder

razaoSocial, CNPJ, segmento, responsavelComercial, telefoneComercial, site, nomeFantasia

Cada tipo de usuário tem campos específicos do seu domínio. O Builder garante que cada perfil seja montado com as regras corretas sem compartilhar lógica desnecessária.

PERFILPRODUTORCONCRETOBUILDER.JAVA — CONSTRUIR() + VALIDAR()

```
@Override
public PerfilProdutor construir() {
    validar();
    return new PerfilProdutor(this);
}

private void validar() {
    if (conta == null)
        throw new IllegalStateException(
            "Conta é obrigatória");
    if (nomePropriedade == null
        || nomePropriedade.isBlank())
        throw new IllegalStateException(
            "Nome da propriedade é obrigatório");
    if (cidade == null || cidade.isBlank())
        throw new IllegalStateException(
            "Cidade é obrigatória");
    if (estado == null
        || estado.length() != 2)
        throw new IllegalStateException(
            "Estado inválido");
}

// getters utilizados pelo construtor de PerfilProdutor
public ContaUsuario getConta() { return conta; }
public String getNomePropriedade() { return nomePropriedade; }
public BigDecimal getAreaHectares() { return areaHectares; }
```


VANTAGENS

Por que Builder é ideal para o AgroPlace

✦ **Legibilidade do código**

Cada campo é nomeado explicitamente na chain fluente. Não há dúvida se o 4º argumento é peso ou quantidade.

✦ **Validação fail-fast**

`construir()` é a única porta de saída. Nenhum objeto inválido chega ao repositório — regras protegidas na camada de domínio.

✦ **Campos opcionais sem `null`**

Campos como `raça` e `descrição` simplesmente não são chamados. Sem sobrecarga de construtores e sem `null` explícito.

✦ **Encapsulamento total do produto**

O construtor de `Anuncio` é `package-private`. Impossível criar um produto malformado fora do Builder.

✦ **Extensibilidade**

Adicionar um campo ao `Anuncio` requer apenas um novo método na interface e no `ConcreteBuilder`. O `Director` e o `Serviço` não mudam — Open/Closed Principle.

✦ **Testabilidade**

Cada `ConcreteBuilder` pode ser testado isoladamente. É possível criar um `MockBuilder` implementando a interface para testes de `Director` sem dependências reais.

✦ **Separação de responsabilidades**

O `Serviço` cuida das regras de negócio. O `Director` cuida do processo de montagem. O `ConcreteBuilder` cuida dos campos e validações.

✦ **Consistência entre módulos**

Os 3 módulos seguem o mesmo padrão. Qualquer desenvolvedor que entra no projeto entende imediatamente a estrutura.

PRINCÍPIOS SOLID

Builder e SOLID — como se complementam

PRINCÍPIO	COMO O BUILDER APLICA NO AGROPLACE	EVIDÊNCIA NO CÓDIGO
S — SRP	Builder cuida exclusivamente da montagem e validação. Serviço cuida da regra de negócio. Director cuida do processo. Cada classe tem uma única razão para mudar.	<code>ServicoAnuncioPadrao</code> não conhece campos de <code>Anuncio</code>
O — OCP	Para adicionar <code>fotoPrincipal</code> ao Anúncio, basta um novo método na interface e no <code>ConcreteBuilder</code> — sem alterar <code>Director</code> ou <code>Serviço</code> .	Interface <code>AnuncioBuilder</code> define contrato extensível
L — LSP	Qualquer novo <code>ConcreteBuilder</code> que implemente <code>AnuncioBuilder</code> pode substituir <code>AnuncioConcretoBuilder</code> no <code>Director</code> sem quebrar o comportamento.	<code>DirectorAnuncio</code> depende apenas da interface
I — ISP	Cada Builder tem apenas os métodos do seu produto. <code>ContaUsuarioBuilder</code> não tem métodos de <code>AnuncioBuilder</code> — interfaces enxutas e coesas.	5 interfaces distintas para 5 entidades
D — DIP	<code>DirectorAnuncio</code> recebe um <code>AnuncioBuilder</code> (interface) no construtor, não um <code>ConcreteBuilder</code> . Depende da abstração, não da implementação.	<code>new DiretorAnuncio(new AnuncioConcretoBuilder())</code>

COMPARATIVO

Antes e depois — impacto real no código

SEM BUILDER

```
new Anuncio(  
  conta,  
  titulo,  
  especie,  
  raca,           // ← null se não tiver raça?  
  quantidade,    // ← parâmetro 5 ou 6?  
  peso,  
  idade,  
  sexo,  
  preco,         // ← e se vier negativo?  
  cidade,  
  estado,        // ← e se tiver 3 caracteres?  
  descricao      // ← null se não tiver?  
);
```

- ✗ Objeto inválido pode ser criado e persistido
- ✗ Troca de posição de parâmetros causa bug silencioso
- ✗ Campos opcionais forçam null explícito
- ✗ Leitura impossível sem IDE

COM BUILDER

```
// Diretor encapsula o processo  
diretor.construirDaRequisicao(  
  conta, req  
);  
  
// internamente — chain fluente:  
builder  
  .conta(conta)  
  .titulo(req.titulo())  
  .especie(req.especie())  
  .precoUnitario(req.precoUnitario())  
  // raça e descrição omitidos se null  
  .construir(); // ← valida tudo aqui
```

- ✓ Validação centralizada — objeto nasce válido
- ✓ Campos nomeados — legibilidade total
- ✓ Campos opcionais simplesmente omitidos
- ✓ Produto inválido nunca sai do Builder

CONCLUSÃO

Builder no AgroPlace — resumo

Problema resolvido

Os 3 módulos criavam objetos com 7 a 12 parâmetros posicionais. O Builder substituiu por uma API fluente nomeada, segura e extensível.

Validação garantida

`validar()` no `ConcreteBuilder` garante que nenhum objeto inválido chega ao `repositorio.save()` — fail-fast antes da persistência.

Arquitetura limpa

Padrão aplicado em sincronia com Clean Architecture — Builder vive no domínio, Director na aplicação, sem vazamento de dependências.

NÚMEROS DO PROJETO

3

módulos com Builder

15

arquivos dedicados ao padrão

5

princípios SOLID atendidos

0

construtores telescópicos no domínio

O padrão Builder é a escolha natural quando um objeto requer muitos parâmetros de inicialização, quando alguns são opcionais e quando a validade do estado é pré-condição crítica para a persistência.